

# LED Control

## LED Control

- 1. GPIO Overview
- 2. Core Concepts
  - 2.1 Common-Anode LED Connection
  - 2.2 Layered Architecture
- 3. ESP32-S3 GPIO Hardware Characteristics
- 4. Project Architecture and Flow
- 5. Hardware Dependencies
- 6. Source Code Analysis
  - 6.1 LED Driver Layer — led.h: Macro Encapsulation
  - 6.2 LED Driver Layer — led.c: Initialization
    - 6.2.1 `gpio_config_t` Structure
    - 6.2.2 `mode` — Six GPIO Operating Modes
    - 6.2.3 `pull_up_en` / `pull_down_en` — Internal Pull-up/Pull-down
  - 6.3 Application Layer — main.c: Program Entry
  - 6.4 Build Files — CMakeLists.txt
- 7. Operation Flow
  - 7.1 Build and Flash
  - 7.2 Expected Behavior
  - 7.3 Verification Methods
- 8. Common Issues

## 1. GPIO Overview

**GPIO (General Purpose Input/Output)** is the most fundamental peripheral interface of MCUs. Through software configuration, chip pins can be set to **Input mode** (reading external signals) or **Output mode** (driving external circuits).

This experiment controls two onboard LEDs to blink synchronously at 1Hz, demonstrating the basic workflow of GPIO output control. The code uses a **component-based layered design**, encapsulating hardware operations into semantic macros, so upper-layer applications don't need to worry about pin numbers or polarity.

Typical GPIO application scenarios:

| Application Type  | Example                                                          |
|-------------------|------------------------------------------------------------------|
| Digital Output    | LED blinking, relay control, buzzer driving                      |
| Digital Input     | Button detection, switch state reading, sensor digital signals   |
| Bidirectional I/O | I2C bus (SDA), single-wire protocols (OneWire / HDQ)             |
| Interrupt Input   | External event triggering (see <b>3. Peripheral Interrupts</b> ) |
| Analog Input      | ADC sampling (GPIO set to DISABLE, driven by ADC)                |

| Application Type    | Example                                                           |
|---------------------|-------------------------------------------------------------------|
| Alternate Functions | UART / SPI / I2C / PWM peripheral signals (configured via IO MUX) |

## 2. Core Concepts

### 2.1 Common-Anode LED Connection

This board uses **Common-Anode** LED connection:

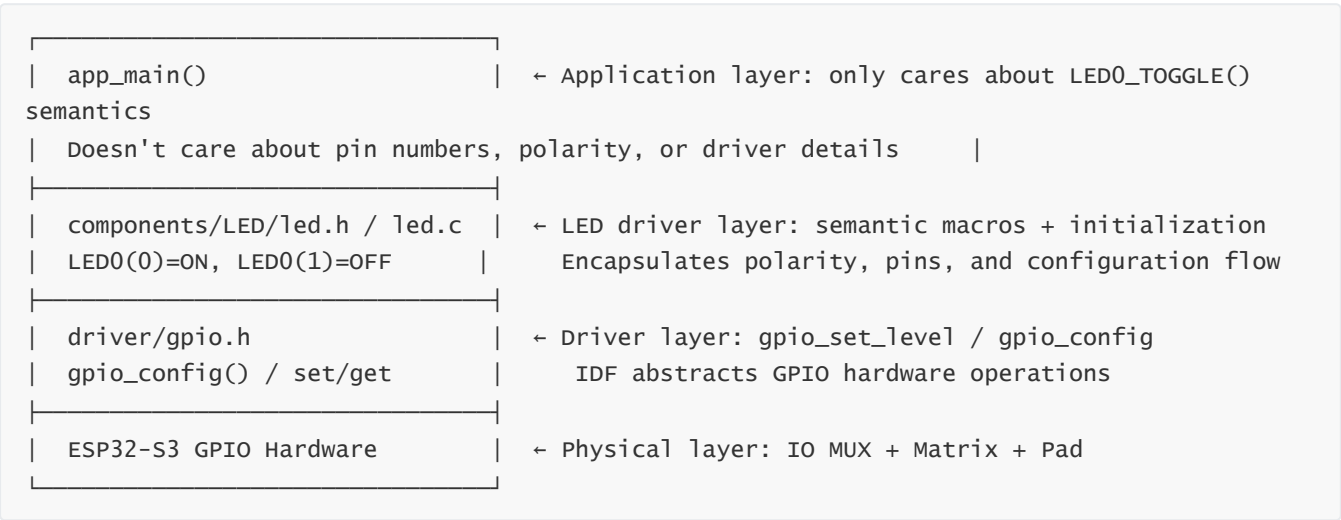


| GPIO Output Level | LED State | Principle                                                                               |
|-------------------|-----------|-----------------------------------------------------------------------------------------|
| Low (0V)          | ON        | Current flows from VDD through LED and resistor into GPIO (source→sink), forming a loop |
| High (3.3V)       | OFF       | GPIO and VDD are at same potential, no current flows through LED                        |

**Core Understanding:** In common-anode connection, GPIO output low = LED ON. This is opposite to the intuition "1=on", so semantic macros are crucial—`LED0(0)` means "turn on LED0", and the macro internally translates to `gpio_set_level(GPIO43, 0)`.

### 2.2 Layered Architecture

This project follows the **hardware abstraction layering** principle:



**Benefits of Layering:**

- 1. Changing pins only requires modifying macros in `led.h`, zero changes to application code
- 2. Semantic macro `LED0(0)` is more intuitive than `gpio_set_level(43, 0)`, eliminating confusion about "why is 0 ON"
- 3. LED driver component is reusable—all example routines needing LED operations can reference

### 3. ESP32-S3 GPIO Hardware Characteristics

| Characteristic             | Parameter                                                                                        |
|----------------------------|--------------------------------------------------------------------------------------------------|
| Available GPIO Count       | ~45 (limited by package and internal multiplexing)                                               |
| Output Drive Current       | Max 40mA (recommended $\leq 20$ mA for long-term use)                                            |
| Input Voltage Range        | 0 ~ 3.3V ( <b>Cannot connect 5V! Level shifting required</b> )                                   |
| Internal Pull-up/Pull-down | ~45k $\Omega$ (weak)                                                                             |
| Output Drive Strength      | 4 levels adjustable (~5 / 10 / 20 / 40 mA)                                                       |
| Interrupt Support          | Edge triggering + level triggering (see 3.NVIC chapter)                                          |
| IO MUX                     | Each pin can be mapped to different peripheral signals (UART/SPI/I2C/PWM, etc.)                  |
| GPIO Exchange Matrix       | On-chip programmable signal routing network, peripheral signals can be mapped to almost any GPIO |
| Direct Output Path         | <code>gpio_set_level</code> bypasses IO MUX and exchange matrix, taking the fastest path         |
| RTC GPIO                   | GPIO0~21 support maintaining level and waking during Deep Sleep                                  |
| Level Hold                 | <code>gpio_hold_en(pin)</code> can lock level during sleep                                       |

#### Special Pin Reminders:

| Pin       | Special                           | Precaution                                                                                   |
|-----------|-----------------------------------|----------------------------------------------------------------------------------------------|
| GPIO0     | Strapping Pin                     | Low during power-on = enters download mode, external circuit should not forcefully pull down |
| GPIO3     | JTAG (TMS)                        | Affects JTAG debugging function during power-on                                              |
| GPIO45/46 | Strapping Pins                    | Affects startup voltage configuration                                                        |
| GPIO26-32 | Connected to internal Flash/PSRAM | Usually unavailable (depends on chip package)                                                |

## 4. Project Architecture and Flow

```
app_main()
├─ nvs_flash_init()      ← Initialize NVS (erase and rebuild if failed)
├─
├─ led_init()            ← Initialize GPIO43 / GPIO44 one by one
├─   ├─ gpio_config(GPIO43)  Push-pull bidirectional, no pull-up/down, no interrupt
├─   ├─ gpio_config(GPIO44)  Push-pull bidirectional, no pull-up/down, no interrupt
├─   ├─ LED0(1)              Initial: output high = OFF
├─   └─ LED1(1)              Initial: output high = OFF
├─
└─ while(1) Main Loop
    ├─ LED0_TOGGLE()        Read level → invert → write
    ├─ LED1_TOGGLE()        Read level → invert → write
    └─ vTaskDelay(500ms)    Delay 500ms (release CPU)
```

Source File Structure:

```
1.LED/
├─ CMakeLists.txt          # Top-level build file
├─ main/
├─   ├─ CMakeLists.txt      # main component (depends on esp_driver_gpio / nvs_flash /
├─   LED)
├─   └─ main.c              # Application entry app_main()
├─ components/
├─   └─ LED/
├─       ├─ CMakeLists.txt  # LED component (depends on esp_driver_gpio)
├─       ├─ led.h           # Pin macros + control macros + function declarations
├─       └─ led.c           # GPIO initialization implementation
```

## 5. Hardware Dependencies

Hardware Connection:

| Pin    | Peripheral | Description                      |
|--------|------------|----------------------------------|
| GPIO43 | LED0       | Common-anode, low level turns on |
| GPIO44 | LED1       | Common-anode, low level turns on |

Dependencies: `esp_driver_gpio`, `nvs_flash`, `freertos`, `LED`

## 6. Source Code Analysis

### 6.1 LED Driver Layer — led.h: Macro Encapsulation

The header file defines pin numbers and semantic control macros, which is the core abstraction of the entire driver:

```
// ===== Pin definitions =====
#define LED0_GPIO_PIN    GPIO_NUM_43
#define LED1_GPIO_PIN    GPIO_NUM_44

// ===== Output control macros (1=OFF, 0=ON) =====
#define LED0(x) do { \
    x ? gpio_set_level(LED0_GPIO_PIN, 1) : \
        gpio_set_level(LED0_GPIO_PIN, 0); \
} while (0)

// ===== Toggle macros =====
#define LED0_TOGGLE() do { \
    gpio_set_level(LED0_GPIO_PIN, !gpio_get_level(LED0_GPIO_PIN)); \
} while (0)
```

**Why wrap macros with `do { ... } while(0)`?** This is the standard safe写法 for multi-statement C macros. Without wrapping, `if (cond) LED0(0); else ...` would cause compilation errors due to semicolon parsing. `do-while(0)` ensures the macro is a complete syntactic unit in any control flow context.

`LED0_TOGGLE()` three-step process:

1. `gpio_get_level()` — Read current level (HIGH=1 / LOW=0)
2. `!` — Invert
3. `gpio_set_level()` — Write inverted level

### 6.2 LED Driver Layer — led.c: Initialization

`led_init()` is the most core function of this experiment—configuring GPIO hardware registers, determining the pin's working mode. Let's break it down line by line and also explain ESP32-S3 GPIO configuration options.

```
void led_init(void)
{
    gpio_config_t gpio_init_struct = {0};

    gpio_init_struct.intr_type      = GPIO_INTR_DISABLE;
    gpio_init_struct.mode           = GPIO_MODE_INPUT_OUTPUT;
    gpio_init_struct.pull_up_en     = GPIO_PULLUP_DISABLE;
    gpio_init_struct.pull_down_en   = GPIO_PULLEDOWN_DISABLE;

    gpio_init_struct.pin_bit_mask   = 1ull << LED0_GPIO_PIN;
    ESP_ERROR_CHECK(gpio_config(&gpio_init_struct));

    gpio_init_struct.pin_bit_mask   = 1ull << LED1_GPIO_PIN;
```

```
ESP_ERROR_CHECK(gpio_config(&gpio_init_struct));

    LED0(1);    // Initial OFF
    LED1(1);    // Initial OFF
}
```

### 6.2.1 `gpio_config_t` Structure

ESP32-S3 GPIO configuration uses **independent struct fields**: mode, pull-up/down, interrupt, and pin are four independent dimensions. Fill in → call `gpio_config()` to write to hardware registers at once. This is more flexible than STM32's traditional approach of encoding 8 modes + pull-up/down into enum combinations.

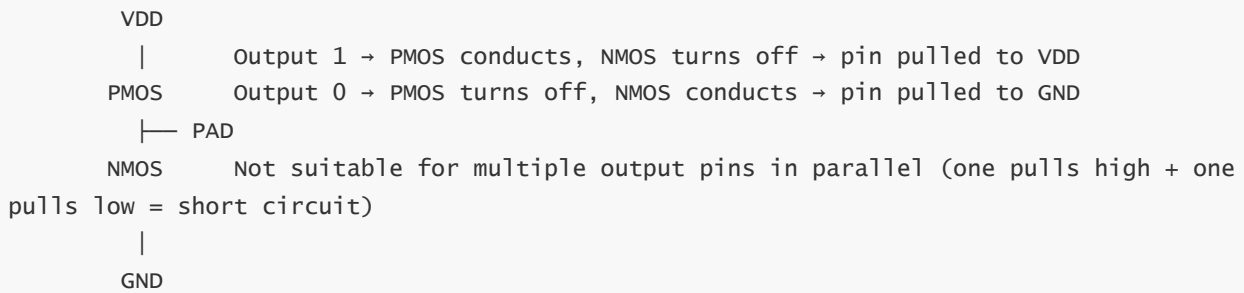
```
typedef struct {
    uint64_t    pin_bit_mask;    // Pin bitmask, bit N=1 means GPIO_N is selected
    gpio_mode_t  mode;           // GPIO mode, 6 choices
    gpio_pullup_t pull_up_en;    // Internal pull-up (0=OFF, 1=ON)
    gpio_pulldown_t pull_down_en; // Internal pull-down (0=OFF, 1=ON)
    gpio_int_type_t intr_type;   // Interrupt type
} gpio_config_t;
```

### 6.2.2 `mode` — Six GPIO Operating Modes

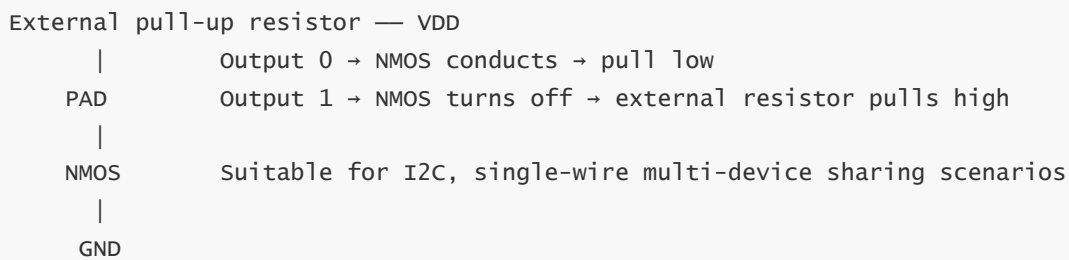
ESP-IDF defines 6 modes. For LED blinking, we only need **two**, the rest are for reference:

| Mode                                   | Input    | Output   | Output Type       | Typical Use                                               |
|----------------------------------------|----------|----------|-------------------|-----------------------------------------------------------|
| <code>GPIO_MODE_DISABLE</code>         | Disabled | Disabled | —                 | Analog input (ADC/DAC), unused pins for power saving      |
| <code>GPIO_MODE_INPUT</code>           | Enabled  | Disabled | —                 | Button reading, digital sensors                           |
| <code>GPIO_MODE_OUTPUT</code>          | Disabled | Enabled  | <b>Push-pull</b>  | LED driving (output only, no readback needed)             |
| <code>GPIO_MODE_OUTPUT_OD</code>       | Disabled | Enabled  | <b>Open-drain</b> | I2C SCL, level shifting                                   |
| <code>GPIO_MODE_INPUT_OUTPUT_OD</code> | Enabled  | Enabled  | <b>Open-drain</b> | I2C SDA, single-wire (OneWire)                            |
| <code>GPIO_MODE_INPUT_OUTPUT</code>    | Enabled  | Enabled  | <b>Push-pull</b>  | LED toggling (needs <code>gpio_get_level</code> readback) |

**Push-Pull Output** — Internally uses PMOS + NMOS complementary pair, high and low levels are actively driven by the chip, fast transition and strong drive capability:



**Open-Drain Output** — Only NMOS, high level relies on external pull-up resistor, multiple devices can share bus (wire-AND):



**This project selects** `GPIO_MODE_INPUT_OUTPUT` (**push-pull bidirectional**): Compared to `GPIO_MODE_OUTPUT`, it adds input enable, ensuring `LED0_TOGGLE()` calling `gpio_get_level()` reliably reads back the current level.

### 6.2.3 `pull_up_en` / `pull_down_en` — Internal Pull-up/Pull-down

ESP32-S3 has built-in ~ **45kΩ** weak pull-up/pull-down resistors:

| Value                                  | Effect                          |
|----------------------------------------|---------------------------------|
| <code>GPIO_PULLUP_DISABLE</code> (0)   | Pull-up disabled                |
| <code>GPIO_PULLUP_ENABLE</code> (1)    | Internal ~45kΩ pull-up to VDD   |
| <code>GPIO_PULLDOWN_DISABLE</code> (0) | Pull-down disabled              |
| <code>GPIO_PULLDOWN_ENABLE</code> (1)  | Internal ~45kΩ pull-down to GND |

Typical combinations:

| PU | PD | Equivalent      | Use Case                                             |
|----|----|-----------------|------------------------------------------------------|
| 0  | 0  | <b>Floating</b> | Output pins—driven by GPIO, no pull-up/down needed   |
| 1  | 0  | Pull-up input   | Button connected to GND, reads high when not pressed |
| 0  | 1  | Pull-down input | Button connected to VDD, reads low when not pressed  |

**This project sets PU=0, PD=0:** LEDs are actively driven by GPIO push-pull output, level is always defined, pull-up/down only adds unnecessary static power consumption.

## 6.3 Application Layer — main.c: Program Entry

```
void app_main(void)
{
    // ===== NVS Initialization =====
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        ESP_ERROR_CHECK(nvs_flash_erase());
        ESP_ERROR_CHECK(nvs_flash_init());
    }

    // ===== LED Initialization =====
    led_init();

    // ===== Main Loop =====
    while (1) {
        LED0_TOGGLE();
        LED1_TOGGLE();
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

**NVS Initialization:** `nvs_flash_init()` initializes ESP-IDF's key-value storage system. Wi-Fi configuration, PHY calibration data, etc. are stored in NVS partition. Even if not using Wi-Fi, initialization is recommended (many IDF subsystems depend on it).

| Error Code                                 | Meaning                                            | Handling         |
|--------------------------------------------|----------------------------------------------------|------------------|
| <code>ESP_OK</code>                        | Success                                            | No action needed |
| <code>ESP_ERR_NVS_NO_FREE_PAGES</code>     | No free pages in partition (common on first flash) | Erase and retry  |
| <code>ESP_ERR_NVS_NEW_VERSION_FOUND</code> | Partition table version changed                    | Erase and retry  |

**Why use `vTaskDelay()` instead of bare-metal `delay()`?** `vTaskDelay()` suspends the current task to Blocked state for 500ms, during which the CPU can execute other tasks. Bare-metal delay is busy-waiting, which should always be avoided under RTOS. `pdMS_TO_TICKS(500)` converts 500ms to FreeRTOS tick count.

## 6.4 Build Files — CMakeLists.txt

**LED Component** (`components/LED/CMakeLists.txt`):

```
idf_component_register(
    SRCS "led.c"
    INCLUDE_DIRS "."
    REQUIRES esp_driver_gpio)
```



- `SRCS` — Component source files
- `INCLUDE_DIRS` — Header file search path (`.` = current directory)
- `REQUIRES` — Depends on `esp_driver_gpio`

**main Component** (`main/CMakeLists.txt`):

```
idf_component_register(  
    SRCS "main.c"  
    INCLUDE_DIRS "."  
    REQUIRES esp_driver_gpio nvs_flash LED)
```

After declaring dependency on `LED`, CMake automatically finds `components/LED/` and links it.

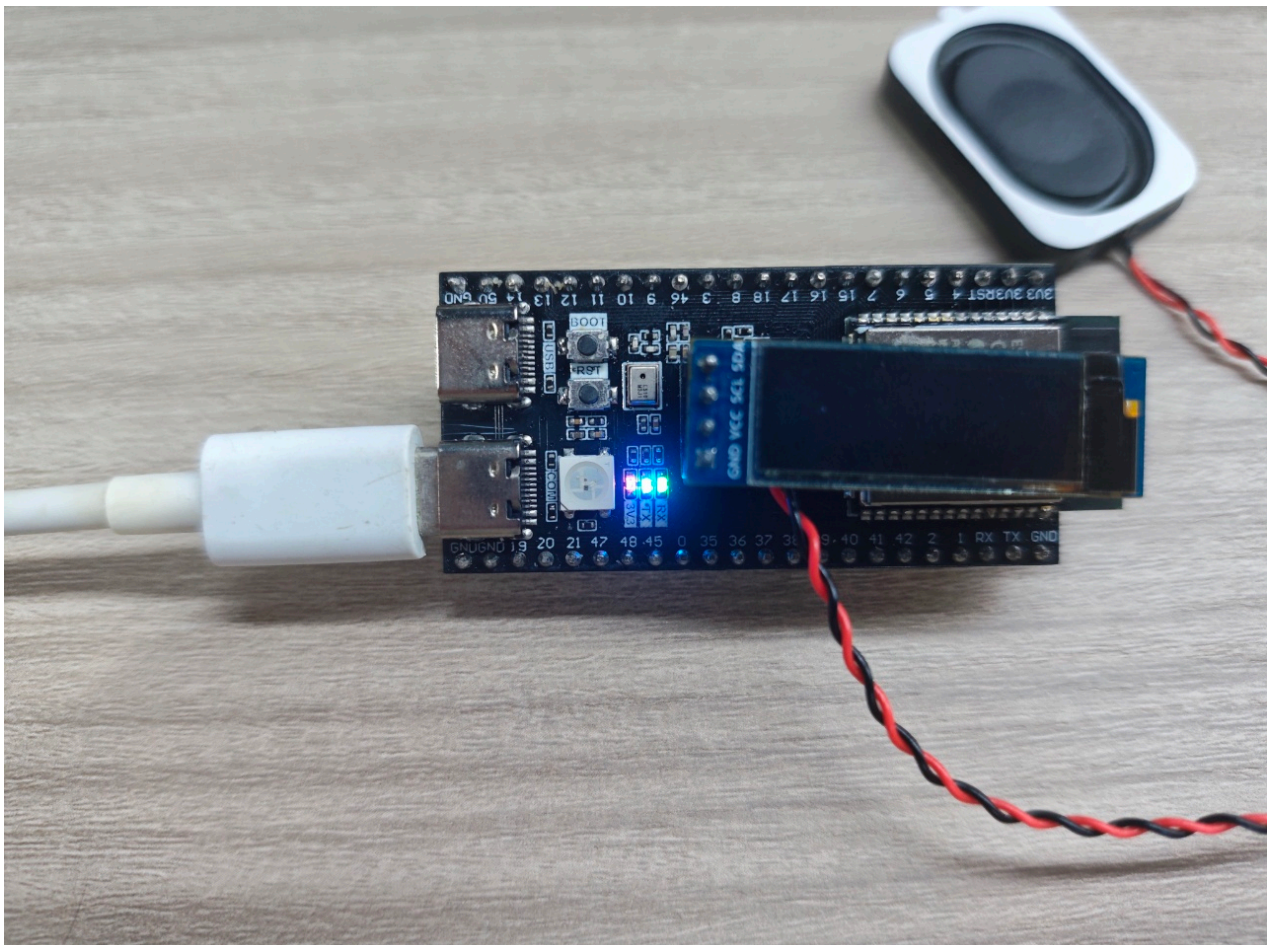
## 7. Operation Flow

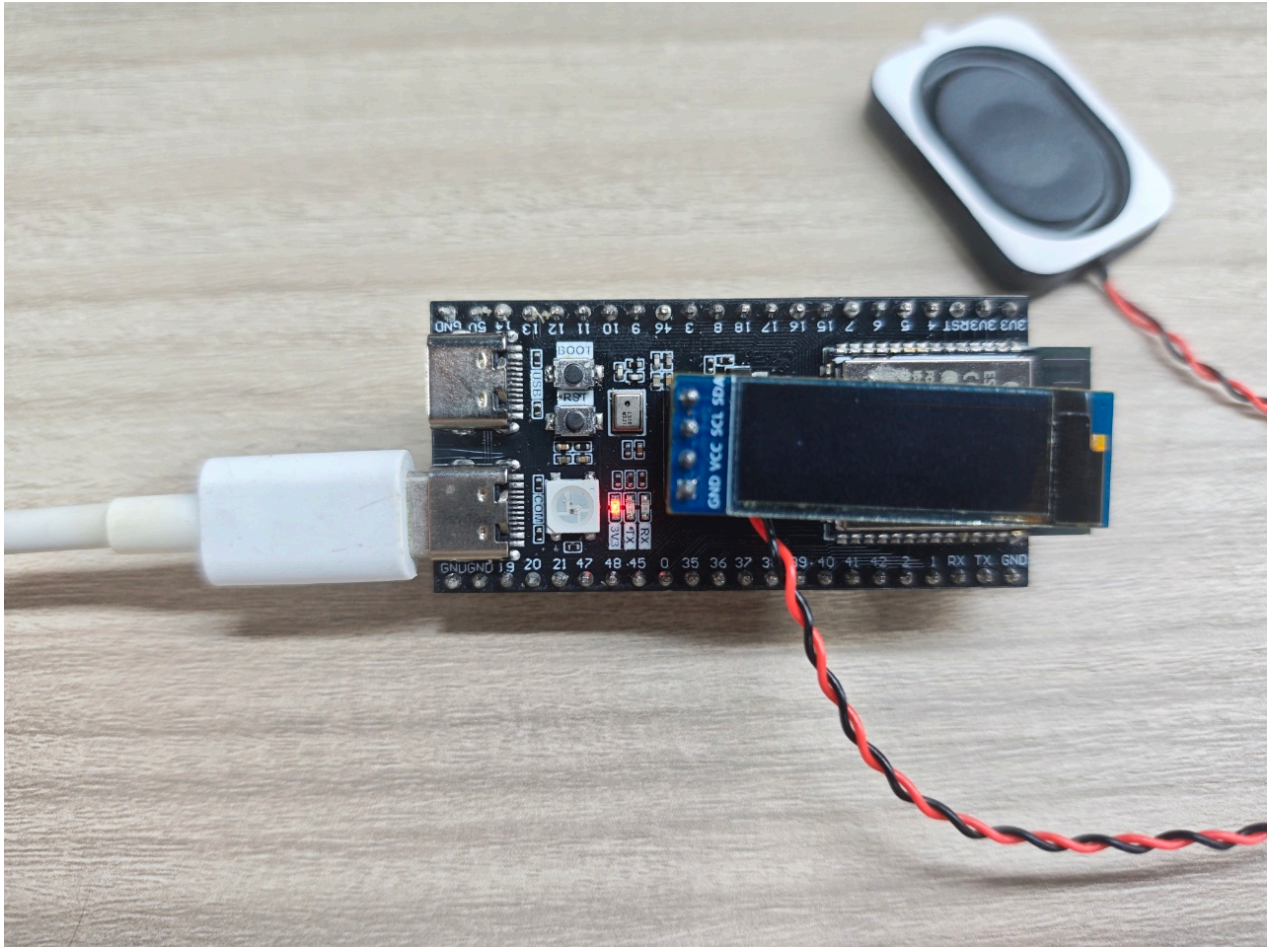
### 7.1 Build and Flash

For detailed flashing process, see: 1. Before Development → 3. Build and Flash

### 7.2 Expected Behavior

- Both LEDs blink **synchronously at 1Hz**
- ON 500ms → OFF 500ms → repeat
- No serial log output (this experiment has no `printf` / `ESP_LOGI` calls)





## 7.3 Verification Methods

| Test Item       | Method                              | Expected Result                |
|-----------------|-------------------------------------|--------------------------------|
| LED0 normal     | Observe GPIO43 LED                  | Stable blinking, period ~1s    |
| LED1 normal     | Observe GPIO44 LED                  | Blinks synchronously with LED0 |
| Frequency check | Time 10 blinks with phone stopwatch | ~5 seconds (10 half-cycles)    |

## 8. Common Issues

| Phenomenon                  | Cause                                                | Solution                                                      |
|-----------------------------|------------------------------------------------------|---------------------------------------------------------------|
| LED doesn't light up        | GPIO configuration error / pin multiplexing conflict | Check if <code>pin_bit_mask</code> matches hardware           |
| LED brightness very low     | GPIO drive strength insufficient                     | Call <code>gpio_set_drive_cap(pin, GPIO_DRIVE_CAP_1)</code>   |
| LED stays on constantly     | Common-anode/common-cathode reversed                 | Confirm hardware connection, adjust macro logic (0↔1)         |
| Doesn't work after flashing | NVS initialization failure causing abort             | First flash automatically erases and rebuilds, this is normal |

| Phenomenon                       | Cause                                                  | Solution                                                                       |
|----------------------------------|--------------------------------------------------------|--------------------------------------------------------------------------------|
| Blinking frequency inaccurate    | <code>vTaskDelay</code> has $\pm 1$ tick jitter        | No effect on LED blinking; use <code>esp_timer</code> for precise timing       |
| No response after program starts | No <code>while(1)</code> loop                          | Under RTOS, system restarts after <code>app_main</code> exits                  |
| GPIO configuration failed        | <code>pin_bit_mask</code> didn't use <code>1u11</code> | <code>1 &lt;&lt; 43</code> overflows, must write <code>1u11 &lt;&lt; 43</code> |